

Team#16 Final Report

Financial Helper Chatbot

Team names

Hun Jang (2019312102) – Team Leader, Full Stack Development (RAG & Streamlit)

Siyul Lee (2021313796) – Data Collection & Pre-processing

Kijeong Roh (2018312350) – Frontend Development (Streamlit)

December 2025

1. Introduction

Motivation

With the rapid digitalization of finance, access to information has become easier, yet the reliability of that information remains a critical issue. While general-purpose Large Language Models (LLMs) like ChatGPT have revolutionized information retrieval, they suffer from significant limitations in the financial domain: they often lack training on the most recent data and are prone to "hallucinations," confidently providing plausible but incorrect information. On the other hand, specialized chatbots employed by commercial banks provide accurate data but are inherently biased, focusing primarily on promoting their own products rather than offering objective financial education.

Financial information demands the highest level of accuracy and neutrality. The lack of a widely available tool that is both accurate (like a specialized bot) and objective (unlike a sales bot) creates a gap in the market. This project was motivated by the need to fill this gap, providing users with a trustworthy AI assistant that enhances financial literacy without commercial bias.

Proposal and Goal

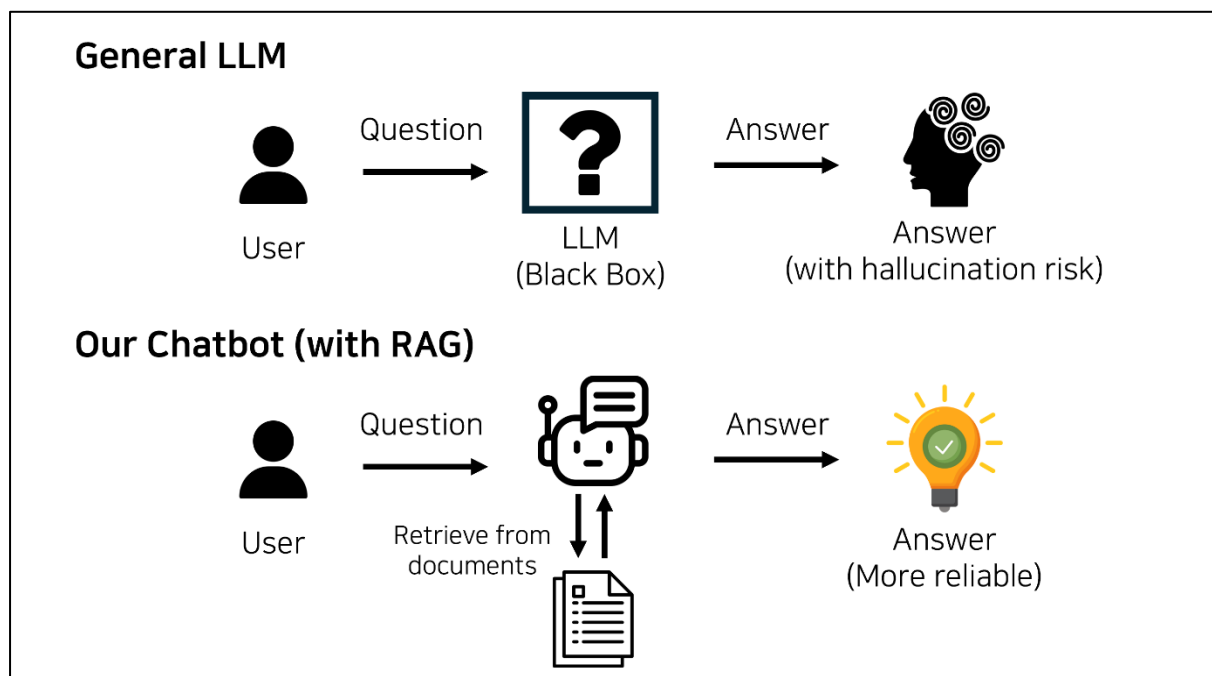
We propose the "**Financial Helper ChatBot**", a web-based conversational AI designed to answer financial questions and provide unbiased advice. To address the hallucination problem of general LLMs, our primary goal is to implement a **Retrieval-Augmented Generation (RAG)** pipeline using the LangChain framework. Instead of generating unverified answers from static memory, the chatbot grounds its responses in a specialized corpus of official

documents, such as the "Financial Terminology Dictionary" and "Financial Tips" from the Financial Supervisory Service (FSS).

The specific objectives of this project are:

1. **Verifiable Accuracy:** To verify the RAG pipeline's effectiveness in delivering accurate information drawn directly from trusted sources.
2. **Source Transparency:** To implement a citation function that provides direct links to the source documents, ensuring users can verify the information.
3. **Accessibility:** To develop a web-accessible prototype that lowers the barrier to complex financial knowledge for general users.

The conceptual difference between a standard LLM and our RAG-based approach is illustrated in **Figure 1**.



[Figure 1] Difference between General LLM and Chatbot with RAG

Existing Approaches

Existing solutions for financial inquiries generally fall into two categories, both of which have distinct limitations:

- **General-Purpose LLMs:** Models like ChatGPT are highly effective at natural language understanding but are unreliable for high-stakes financial advice due to the risk of hallucinations and outdated knowledge.
- **Institutional Chatbots:** Chatbots from financial institutions possess accurate data but are limited to their own services. Their primary function is customer service and sales, making them unsuitable for users seeking comparative or objective financial knowledge.

Our approach differs by combining the strengths of both: utilizing the reasoning capabilities of LLMs while restricting their knowledge base to a curated, objective dataset. This ensures the system acts as a neutral "Financial Secretary" rather than a salesperson.

Project main components

The final system comprises three core components:

- **Frontend Interface:** Built with **Streamlit**, offering a user-friendly chat interface with features for session management and onboarding guidance. (Originally planned as a Flask web server, we adopted Streamlit to focus on rapid prototyping of data interactions).
- **RAG Backend:** Orchestrated by **LangChain**, this component manages the flow of retrieving relevant document chunks from the vector store and prompting the

OpenAI GPT model to generate answers based solely on that context.

- **Knowledge Base:** A vectorized database (**FAISS**) created from a specialized corpus of objective financial documents (TXT, PDF), ensuring the chatbot's advice is grounded in official data.

2. Project Design and Implementation

Main Features

- **User Feature**

The user interface is designed to be accessible to anyone, regardless of their technical expertise.

- **Intuitive Chat Interface:** A familiar chat-based UI allows users to ask questions in natural language.
- **Onboarding Guidance:** "Recommended Question" chips (e.g., "How to check dormant deposits?") are displayed on the initial screen to help first-time users start a conversation effortlessly.
- **Session Management:** A robust sidebar allows users to manage their API keys securely and browse or delete past conversation histories, providing a personalized experience.

- **Main Feature**

The core engine of our system is the **Retrieval-Augmented Generation (RAG) Pipeline**.

- **Context-Aware Generation:** Unlike standard LLMs that rely on static training data, our system first retrieves relevant documents from a local vector store (FAISS) constructed from official financial data.
- **Hybrid Prompting Strategy:** We implemented a sophisticated prompting logic. If the retrieved context contains a dictionary definition, the model prioritizes it. If not, it falls back to general knowledge but appends any "useful tips" found in the context, ensuring the answer is always helpful and grounded.

- **Extension Feature**

We implemented advanced features to maximize reliability and usability, distinguishing our project from simple demos.

- **Smart Source Linking:** To ensure verifiable accuracy, the system provides dynamic hyperlinks to the original sources. For financial tips (c_*.pdf), we reverse-engineered the FSS bulletin board's numbering logic to generate direct URLs to the specific posts. For terminology (a_*.txt), links direct users to the search results on the FINE portal.
- **Persistent Chat History:** Leveraging Python's pickle module, we built a local storage system that saves conversation sessions. This allows users to maintain context across multiple turns and revisit previous consultations even after restarting the browser.

Technical Stacks

We selected a technology stack that balances performance with rapid prototyping capabilities.

- **Frontend Framework:** Initially, we planned to use **Flask** with HTML/CSS to build a custom web server. However, to focus more on the core RAG logic and achieve rapid iteration of data visualization, we strategically shifted to **Streamlit**(v1.34.0+). This allowed us to build a polished, interactive data application entirely in Python without the overhead of full-stack development.
- **Backend (RAG Orchestration):** **LangChain** (v0.2.17) manages the complex flow of document loading, splitting, embedding, and retrieval.

- **AI Model: OpenAI GPT-4o-mini** serves as the reasoning engine, selected for its cost-effectiveness and high performance in Korean language tasks.
- **Database (Vector Store): FAISS (Facebook AI Similarity Search)** provides efficient similarity search for our document chunks. We implemented a pre-build script (`build_vectorstore.py`) to index data offline, ensuring fast startup times.
- **Deployment & Tools:** The project is version-controlled via **Git/GitHub**. For dependency management, we primarily used **pip**, while **Conda** was utilized specifically for macOS (Apple Silicon) environments to resolve C++ build issues related to the FAISS library.

Expected Challenges

At the initial planning stage, we identified three major challenges that could impact the project's success:

- **Complexity of RAG Implementation:** Since Retrieval-Augmented Generation (RAG) involves complex orchestration between LLMs, vector databases, and retrieval logic, we anticipated a steep learning curve. In particular, understanding and utilizing the **LangChain** framework—which abstracts these processes—was expected to be difficult due to our team's limited prior experience with advanced AI frameworks.
- **Frontend-Backend Integration:** Integrating a Python-based AI model with a traditional frontend (like React or Vue) usually requires building complex REST APIs and managing asynchronous data flows. We were concerned that handling this full-stack integration would consume too much time and resources, potentially diverting focus from the core AI logic. (This concern was later mitigated by adopting

Streamlit).

- **Securing High-Quality Data:** Before starting data collection, we were uncertain about the availability of "clean" and reliable financial data. We anticipated that public data might be unstructured, scattered, or in formats difficult to parse (e.g., image-based PDFs), which would pose a significant hurdle for building an accurate vector store.

3. Design for the Proposed System

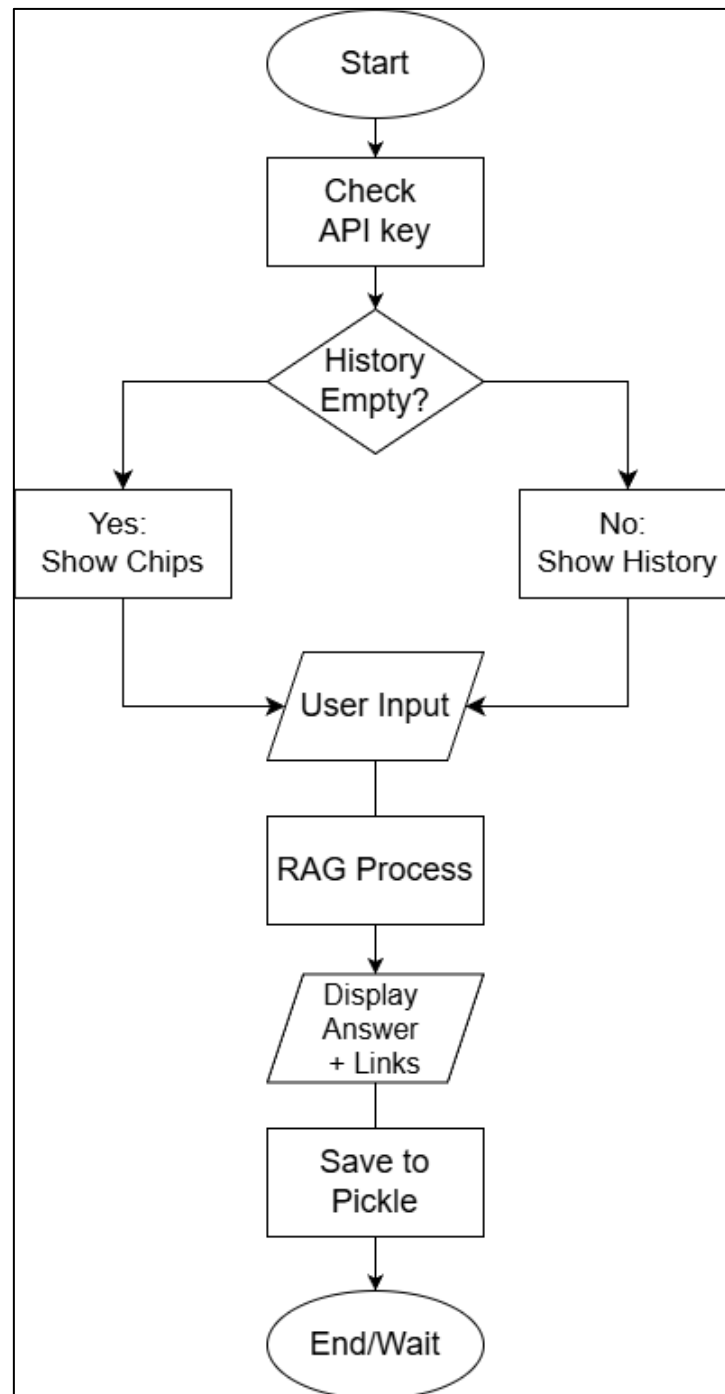
Overall Architecture

- **Frontend Flowchart**

The user interaction flow is designed to be linear yet flexible:

1. **Initialization:** The app loads and checks for a valid API Key in the session state.
2. **Onboarding:** If the chat history is empty, "Recommended Question" chips are displayed.
3. **Input Processing:** When a user inputs a query (via text or button click), the system captures the input and triggers the RAG pipeline.
4. **Response Generation:** The system displays a loading spinner while the backend processes the query.
5. **Display & Link:** The generated answer is rendered, followed by "Smart Source Links" that direct users to the specific FSS/FINE web pages.
6. **History Saving:** The interaction is automatically serialized and saved to a local pickle file.

The detailed user interaction flow is depicted in **Figure 2**.



[Figure 2] Frontend Flowchart

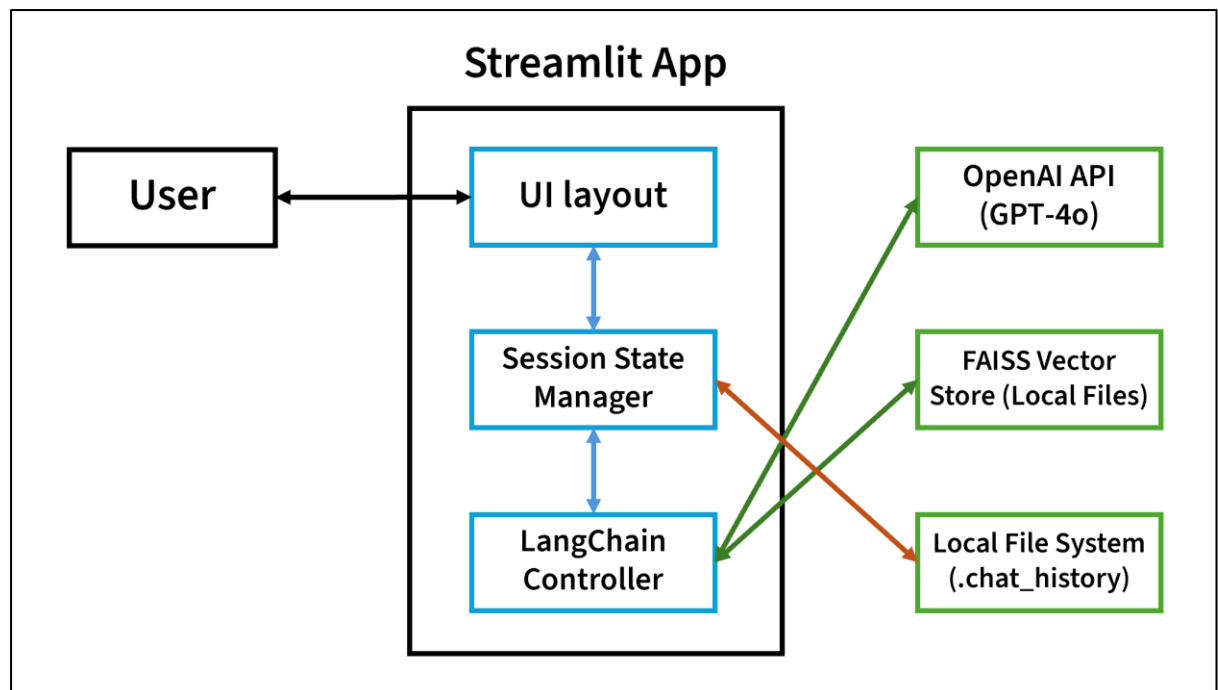
- **Backend API Specification**

Since the application is built as a monolithic Streamlit app, internal Python functions serve as the API endpoints. Key functions include:

- `get_rag_chain(api_key)`: Initializes the LangChain retrieval pipeline.
 - **Input:** User's OpenAI API Key.
 - **Output:** A ready-to-execute RetrievalQA chain object.
- `load_vectorstore()`: Loads the pre-indexed FAISS database from the local file system.
 - **Optimization:** Cached via `@st.cache_resource` to prevent reloading on every interaction, significantly reducing latency.
- `process_response(user_input)`: Orchestrates the generation and storage process.
 - **Logic:** Invokes the chain -> Parses metadata for source linking -> Appends to session state -> Triggers `save_chat_history()`.

- **Architecture diagram**

The overall system architecture, connecting the user, Streamlit app, and external resources, is shown in **Figure 3**.



[Figure 3] Architecture diagram

General Challenges

- **Frontend**

Developing a polished UI with Streamlit presented specific layout challenges, which we resolved through custom CSS injection and state management:

- **Layout Instability:** The default Streamlit sidebar had excessive top padding due to the header element, and the chat input bar was not aligned with the main content width. We resolved this by reverse-engineering the DOM structure and applying custom CSS to specific data-testid elements (e.g., stSidebarUserContent, stChatInput), ensuring a cohesive and professional look.
- **State Management:** Handling dynamic UI elements, such as hiding the "Recommended Question" buttons immediately after the first interaction, required precise control of the application lifecycle. We utilized st.rerun() strategically within callback functions to force UI updates and ensure the interface reflected the current state instantly.

- **Backend**

The backend implementation involved overcoming several technical hurdles related to environment compatibility and API constraints:

- **Dependency Conflicts (FAISS):** Installing faiss-cpu on macOS (Apple Silicon) caused C++ compilation errors due to missing build tools (swig, libomp). We resolved this by utilizing **Conda** to install pre-built binaries from conda-forge, while using standard pip for Windows environments.
- **Token Limit Exceeded:** During the vector store creation, attempting to embed all

documents in a single batch triggered OpenAI's `BadRequestError` (token limit exceeded). We solved this by implementing a batching mechanism (`chunk_size`) in the embedding process to send document chunks in manageable groups.

- **Library Versioning:** The transition of LangChain from v0.1 to v0.2 caused `ModuleNotFoundError` due to changes in import paths (e.g., `langchain_community`). We refactored the codebase to adhere to the latest modular architecture and strictly pinned versions in `requirements.txt` to ensure reproducibility.

4. Implementation

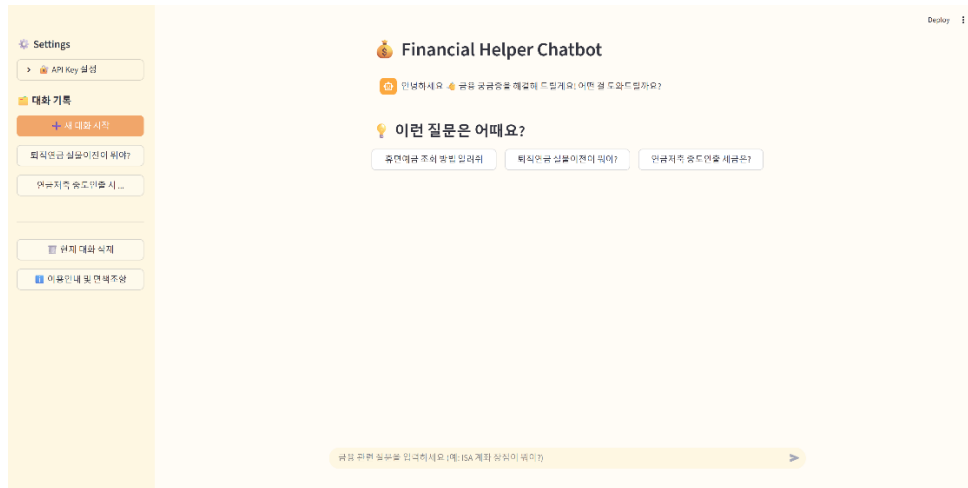
UI Design Process (Instead of Figma Design)

Instead of relying on static wireframes, we adopted an **Iterative Design Process** using Streamlit's rapid prototyping capabilities. We started with a basic chat interface and progressively refined the UI/UX based on team feedback.

- **Phase 1 (Basic):** A simple input/output interface focused on testing the RAG pipeline connection.
- **Phase 2 (Theming):** Applied a "Financial Gold & Cream" color scheme (`.streamlit/config.toml`) to convey trust and wealth.
- **Phase 3 (UX Enhancement):** Added "Onboarding Chips" for first-time users and a comprehensive "Sidebar" for session management, ensuring a professional application feel.

Main screens snapshots

The initial user interface is designed for accessibility as seen in **Figure 4**, while **Figure 5** demonstrates the RAG response with verification links.



[Figure 4] **Main Chat Interface** The initial screen featuring the financial theme, API key setup, and onboarding chips to guide first-time users.



[Figure 5] **RAG Response with Smart Links** A demonstration of the chatbot providing an answer based on retrieved documents, accompanied by direct hyperlinks to the official FSS/FINE pages.

Code snapshots

Snippet 1. Hybrid System Prompt

This prompt logic ensures the chatbot prioritizes dictionary definitions when available, while falling back to general knowledge with useful tips for broader queries.

```
system_prompt = (  
  
    "1. **(Search Definition)**: First, check if there is a dictionary definition in  
    the {context}.\n"  
  
    "2. **(Case 1: Found)**: If found, provide the core answer based on the  
    definition.\n"  
  
    "3. **(Case 2: Not Found)**: If not found, explain using general knowledge but  
    append any 'Useful Tips' found in the context."  
  
)
```

```
system_prompt = (  
    "너는 금융 정보를 설명하는 전문 도우미야. 너의 답변은 다음 규칙을 따라야 해:\n\n"  
    "---- 답변 규칙 ----\n"  
    "1. **(정의 검색)**: 먼저 '참고 자료'({context})에서 질문받은 용어의 **사전적 정의**가 있는지(주로 'a_*.txt' 파일) 확인해.\n"  
    "2. **(Case 1: 정의 있음)**: 만약 '참고 자료'에 사전적 정의가 있다면:\n"  
    "   가. 해당 정의를 바탕으로 **핵심 답변**을 해. [cite]를 꼭 달아줘.\n"  
    "   나. '참고 자료'에 추가적인 **유용한 정보** (꿀팁, 예시 등, 주로 'c_*.pdf' 파일)가 있다면 '유용한 추가 정보:' 섹션을 만들어 요약해줘.\n"  
    "3. **(Case 2: 정의 없음)**: 만약 '참고 자료'에 사전적 정의가 **없다면** (예: '적금'):\n"  
    "   가. 너의 **일반 지식**을 사용해 해당 용어의 정의를 설명해.\n"  
    "   나. **하지만**, '참고 자료'에 그 용어와 관련된 **유용한 정보** (꿀팁, 예시 등)가 검색되었다면, '참고 자료에서 찾은 유용한 정보:' 섹션을 만들어 반드시 요약해줘.\n"  
    "4. **(무관한 자료)**: 만약 '참고 자료'가 질문과 **전혀 무관**하다면, 무시하고 너의 일반 지식으로만 답해.\n"  
    "5. **(필수 안내)**: 모든 답변 마지막에는 항상 아래의 안내 문구를 추가해.\n"  
    "   * 본 정보는 참고용 일반 설명입니다. 실제 투자/세무/법률 판단은 공신력 있는 최신 자료와 전문가 상담을 권장합니다. '\n"  
    "---- 참고 자료 ----\n"  
    "{context}"  
    "----"  
)
```

[Snippet 1] Hybrid System Prompt Logic

Snippet 2. Smart Source Linking Logic

This function parses the filename (e.g., c_000.pdf) and generates a direct URL to the specific post on the Financial Supervisory Service's bulletin board using a reverse-mapping algorithm.

```
def display_source_item(source, search_query=""):
    # ... (Path cleaning omitted)
    elif "FinancialTips" in source:
        # Reverse calculation: c_000 corresponds to post #157
        tip_number = 157 - file_num

        board_url = f"https://www.fss.or.kr/...&searchWrd=- ({tip_number})"

        st.markdown(f"- 💡 **[Go to Financial Tip #{tip_number}]({board_url})**")
```

```
def display_source_item(source, search_query=""):
    clean_source = source.replace("\\", "/")
    file_name = clean_source.split("/")[-1]

    if "FINE금융용어사전" in source or file_name.endswith(".txt"):
        try:
            name_body = file_name.replace(".txt", "")
            parts = name_body.split("_")

            if len(parts) >= 3:
                term = "_".join(parts[2:])
            else:
                term = name_body
        except:
            term = file_name

        base_url = "https://fine.fss.or.kr/fine/fncTip/fncDicary/list.do?menuNo=900021"
        search_url = f"{base_url}&searchCnd=2&searchStr={term}"
        st.markdown(f"- 📖 **[FINE 금융용어사전: '{term}]({search_url})**")

    elif "금융꿀팁" in source or ("c_" in file_name and ".pdf" in file_name):
        try:
            file_num_str = file_name.replace("c_", "").replace(".pdf", "")
            file_num = int(file_num_str)

            tip_number = 157 - file_num

            search_query_encoded = urllib.parse.quote(f"- ({tip_number})")
            board_url = f"https://www.fss.or.kr/fss/bbs/B0000173/list.do?menuNo=200498&searchCnd=1&searchWrd={search_query_encoded}"

            st.markdown(f"- 💡 **[금융꿀팁 {tip_number}호 (금융감독원)]({board_url})**")

        except Exception:
            st.write(f"- {source}")

    else:
        st.write(f"- {source}")
```

[Snippet 2] Smart Source Linking Function

Snippet 3. Offline Vector Store Construction

To eliminate latency during the user's initial access, we separated the document embedding process. This script pre-processes PDF/TXT files, splits them into chunks, creates a FAISS index, and serializes it to disk. The main application then simply loads this pre-built index.

```
def build_vectorstore():
    # ... (Loaders setup omitted)

    text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=100)

    splits = text_splitter.split_documents(all_docs)

    # Optimization: Batch embedding to avoid Token Limit Error
    embeddings = OpenAIEmbeddings(api_key=api_key, chunk_size=200)
    vectorstore = FAISS.from_documents(splits, embeddings)

    vectorstore.save_local("faiss_index") # Save to disk
```

```

def build_vectorstore():
    import streamlit as st
    print("벡터 스토어 빌드를 시작합니다...")

    # 1. API 키 확인
    api_key = st.secrets.get("openai_api_key", None)
    # 2) if not exist, find OPENAI_API_KEY
    if not api_key:
        api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        print("오류: OPENAI_API_KEY 환경 변수가 설정되지 않았습니다.")
        print(".env 파일을 생성하거나 터미널에서 직접 키를 설정해주세요.")
        return

    # 2. 문서 경로 정의
    pdf_path = "./data/금융감독원금융폴립200선"
    txt_path = "./data/FINE금융용어사전"

    # 3. 로더 생성 및 문서 로드
    print(f"'{pdf_path}'에서 PDF 문서를 로드합니다...")
    pdf_loader = PyPDFDirectoryLoader(pdf_path)
    pdf_docs = pdf_loader.load()

    print(f"'{txt_path}'에서 TXT 문서를 로드합니다...")
    txt_loader = DirectoryLoader(
        txt_path,
        glob="**/*.txt",
        loader_cls=TextLoader,
        loader_kwargs={"encoding": "utf-8"}
    )
    txt_docs = txt_loader.load()
    all_docs = pdf_docs + txt_docs
    print(f"총 {len(all_docs)}개의 문서를 로드했습니다.")

    # 4. 문서 분할 (Chunking)
    print("문서를 분할합니다 (Chunking)...")
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
    splits = text_splitter.split_documents(all_docs)
    print(f"총 {len(splits)}개의 문서 조각으로 분할되었습니다.")

    # 5. 임베딩 모델 생성
    print("OpenAI 임베딩 모델을 사용해 벡터화를 시작합니다... (시간이 걸립니다)")
    embeddings = OpenAIEmbeddings(api_key=api_key, chunk_size=200)

    # 6. 벡터 스토어 생성 (FAISS) 및 저장
    start_time = time.time()
    vectorstore = FAISS.from_documents(splits, embeddings)
    end_time = time.time()

    print(f"벡터화 완료. (소요 시간: {end_time - start_time:.2f}초)")

    # 7. 로컬에 저장
    vectorstore.save_local(VECTORSTORE_PATH)
    print(f"성공: 벡터 스토어를 '{VECTORSTORE_PATH}' 폴더에 저장했습니다.")

```

[Snippet 3] Offline Vector Store Construction Script

Debugging and Testing

We conducted rigorous debugging to ensure system stability across different environments and UI scenarios.

- **CSS Layout Debugging:** We encountered an issue where the Streamlit sidebar header consumed excessive vertical space (931px), pushing content off-screen. Using browser Developer Tools, we identified the `stSidebarHeader` class as the culprit and injected custom CSS (`display: none` or `height: auto`) to fix the layout.
- **Cross-Platform Compatibility:** The `faiss-cpu` library failed to build on macOS (Apple Silicon) due to missing C++ compilers (swig). We successfully resolved this by creating a dedicated Conda environment to install pre-built binaries, while confirming standard pip installation worked for Windows.
- **Source Link Verification:** We manually verified the "Smart Links" by clicking generated URLs for various documents (e.g., `c_000.pdf` to `c_155.pdf`) to ensure they correctly landed on the intended FSS bulletin board posts.

5. Expected Result

Upon the successful deployment of the Financial Helper Chatbot, we achieved the following measurable outcomes:

- **Functional Prototype:** A fully operational web-based chatbot accessible via any browser, featuring a professional financial-themed UI.
- **High Accuracy & Verification:** Users receive answers grounded in official FSS documents. Crucially, the "Smart Link" feature allows users to click and verify the source instantly, bridging the gap between AI convenience and factual reliability.
- **User Retention:** With the implementation of the conversation history feature, the system successfully retains context across sessions, transforming a one-off query tool into a continuous financial assistant.

6. Limitations and Discussions

While the developed prototype successfully demonstrates the feasibility of a reliable financial RAG chatbot, we identified several structural limitations inherent to its current pre-deployment stage:

1. Infrastructure & Scalability (Deployment Constraints)

- **Limitation:** The current system operates as a local prototype running on a single Streamlit instance. It lacks a dedicated server infrastructure (e.g., NGINX, Load Balancer) to handle high-concurrency input/output (I/O) requests.
- **Discussion:** In a real-world production environment with multiple simultaneous users,

the current single-threaded execution model would lead to significant latency or service failure. Future work must involve deploying the application to a cloud environment (e.g., AWS, Azure) and implementing an auto-scaling architecture.

2. Data Persistence (Pickle vs. Database)

- **Limitation:** To prioritize rapid prototyping, we implemented session history storage using Python's pickle serialization on the local file system.
- **Discussion:** While efficient for a single-user demo, file-based storage is unsuitable for production due to concurrency issues (file locking), lack of query capabilities, and potential security vulnerabilities. A production-ready system should migrate to a robust relational database (e.g., PostgreSQL) or NoSQL database (e.g., MongoDB) to manage user sessions and chat history securely and efficiently.

3. Limited Scope of Knowledge Base

- **Limitation:** The current RAG vector store is constructed solely from the "Financial Terminology Dictionary" and "Financial Tips 200."
- **Discussion:** While highly reliable, this static dataset limits the chatbot's ability to answer broader financial questions involving real-time market data, changing interest rates, or complex tax laws. Future iterations should expand the data pipeline to ingest a wider range of authorized financial documents and potentially integrate external APIs for real-time information retrieval.

7. Related Works

Our project builds upon and distinguishes itself from existing solutions:

- **vs. Commercial Bank Chatbots:** While bank chatbots (e.g., KB StarBanking, Shinhan Sol) are robust, they function primarily as navigation tools for banking apps and product sales. Our chatbot differs by focusing purely on **objective financial literacy and education**, utilizing public data rather than commercial product data.
- **vs. General LLMs (ChatGPT):** Standard LLMs often hallucinate when asked about specific Korean financial regulations. By applying the **RAG architecture**, we significantly reduced hallucination risks, ensuring that the model refuses to answer or provides a disclaimer when the information is not found in the trusted corpus.

8. Conclusion

This project successfully demonstrated that a **Retrieval-Augmented Generation (RAG)** system can effectively solve the reliability issues of general LLMs in the specialized domain of finance. By strategically choosing **Streamlit** for rapid development and **LangChain** for robust AI orchestration, we built a "Financial Helper Chatbot" that is not only accurate but also user-friendly.

The inclusion of features such as **Smart Source Linking** and **Session History Management** elevated the project from a simple technical demo to a practical service prototype. We believe this system contributes to lowering the barrier to financial information, helping users make informed decisions based on trustworthy public data.

Our website link: <https://capstone-financial-chatbot.streamlit.app/>

OpenAI key:

9. References

- Financial Supervisory Service (FSS), "Financial Tips 200" & "FINE Portal".
- LangChain Documentation (v0.2).
- Streamlit Documentation.
- Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks."